
120 minutes. Tous Documents papier autorisés. Barème à titre indicatif.

1 Esterel

Un robot se déplace avec deux chenilles (une à gauche G , une à droite D) ainsi :

- pour avancer, G et D s'activent en marche avant.
- pour reculer, G et D s'activent en marche arrière.
- pour tourner sur place vers la gauche, G s'active en marche avant et D en marche arrière. (symétriquement pour tourner sur place vers la droite)
- pour tourner vers la gauche en avançant, G s'active en marche avant et D ne s'active pas. (symétriquement pour tourner vers la droite en avançant)
- pour tourner vers la gauche en reculant, D s'active en marche arrière et G ne s'active pas. (symétriquement pour tourner vers la droite en reculant)
- le moteur peut être éteint (les chenilles s'arrêtent et ne répondent plus) et (ré-)allumé.

Le système Esterel comporte envoi des signaux aux deux chenilles et reçoit des ordres d'un pilote selon quatre signaux : un pour avancer/reculer, un pour tourner à gauche/à droite, un pour éteindre le moteur, et un pour l'allumer.

1. Faire un schéma d'un système en listant les composants, les signaux d'entrées et les signaux de sorties.
2. Ecrire le système Esterel.
3. Réécrire le système pour introduire un deuxième pilote, le robot fait "la moyenne" des ordres des deux pilotes :
 - tourner à gauche + tourner à droite = ne pas tourner
 - tourner à gauche + neutre = tourner à gauche
 - tourner à gauche + tourner à gauche = tourner à gauche
 - ...

2 Web

On cherche à développer une *plat-forme d'examen des candidatures à des formations* comprenant les fonctionnalités suivantes :

- des *candidats* peuvent se connecter sur la plateforme, accéder à une liste de formations, choisir jusqu'à 10 formations, et, pour chacune d'entre elle, remplir un formulaire de candidature et déposer des pièces (relevé des notes, diplômes, CV, lettre de motivation, ...)
- des *examineurs*, liées à une ou plusieurs formations, peuvent se connecter sur la plateforme, accéder à la liste des candidats ayant déposé un dossier pour les formations auxquels ils sont liés, consulter les dossiers et les évaluer.
- les dossiers de candidatures à une formation sont soit rejetés, soit classés (par ordre de pertinence des candidatures au sein d'une même formation).

- l'utilisation de la plateforme se déroule en 3 phases successives qui ne se superposent pas : 1) les candidats déposent des candidatures, 2) les examinateurs examinent les dossiers, 3) un algorithme (on ne s'occupera pas des détails de cet algorithme dans cet exercice) appelle chaque jour, dans chaque formation, des candidats ; ceux-ci peuvent accepter (un unique appel) ou rejeter cet appel, laissant leur place aux candidats suivants dans le classement.
- dans la phase 2, plusieurs examinateurs d'une même formation peuvent accéder aux dossiers des candidats, et peuvent voir et synchroniser leurs évaluations respectives.

Le but de cet exercice est de réaliser un mini-dossier décrivant l'implémentation de cette application web en respectant les points suivants:

- l'application est composée d'un unique client pour les candidats et les examinateurs et d'un unique serveur.
- Le serveur Web est écrit en Servlets purs ou en Go (sans utiliser de *framework* de développement).
- le client et le serveur communiquent selon une API REST.

Le dossier comprend:

1. **Schéma:** un schéma général de l'application, détaillant les composants internes coté client et côté serveur, les différentes communications entre les composantes de l'application, et une répartition des fonctionnalités.
2. **Use Cases:** une série de plusieurs *use cases* décrivant **textuellement** (en prose) des exemples d'utilisations caractéristiques de l'application.
3. **DB:** une description des bases de données, des différentes tables/documents et de leur composition ainsi que quelques exemples d'entrées.
4. **REST:** une description complète de l'API REST mise en oeuvre par le serveur, des mots-clés utilisés, des requêtes possibles, ainsi que plusieurs exemples d'utilisation (requête HTTP, réponse HTTP).
5. **Servlets/Handlers:** un plan (noms, signatures, rôles) des Servlets/Handlers présents dans le serveur de l'application, de leurs liens avec les mots-clé de l'API ou les services.
6. **Servlet:** le plan du code d'un Servlet ou d'un *handler* Go caractéristique, au choix.
7. **Client:** le plan du site pour les utilisateurs avec les différents écrans proposés (on peut choisir un client monopage), et une description des contenus proposés et la mise en avant des appels asynchrones (AJAX) effectués par le client.
8. **Javascript:** le code JS de la partie du client chargée d'afficher, chaque jour, les formations ayant appelé le candidat connectés..

3 Lustre

Ecrire en Lustre :

1. un noeud qui prend en entrée un flot d'entiers $(x_n)_{n \in \mathbb{N}}$ et qui renvoie en sortie un flot d'entiers $(y_n)_{n \in \mathbb{N}}$ défini par $y_0 = 0$, et $y_{n+1} = \begin{cases} 1 & \text{si } x_{n+1} > x_n \\ 0 & \text{si } x_{n+1} = x_n \\ -1 & \text{si } x_{n+1} < x_n \end{cases}$.
2. un noeud qui prend en entrée un flot d'entiers $(x_n)_{n \in \mathbb{N}}$, un flot de booléens $(b_n)_{n \in \mathbb{N}}$ et qui renvoie un flot d'entier $(y_n)_{n \in \mathbb{N}}$ tel que y_n est la somme de tous les x_i pour $0 \leq i \leq n$ tels que b_i est vrai (i.e. la sortie y est la somme de toutes les entrées x précédentes qui ont eu lieu à un instant où l'entrée b était vraie)

4 Canaux Synchrones

On modélise un système composé d'un serveur et de plusieurs clients réalisant un mécanisme d'inscription/publication :

- Le serveur peut être contacté sur un canal public pour une *inscription*. Il ajoute un canal (fourni dans la requête) à une liste d'abonné.
- Le serveur peut être contacté sur un autre canal public pour une *diffusion*. Il envoie un message (fourni dans la requête) à tous les canaux abonnés.
- Le serveur peut être contacté sur un autre canal public pour une *désinscription*. Il retire un canal (fourni dans la requête) à sa liste d'abonnés.

1. Faire un schéma du système.
2. Ecrire le système en Go ou OCaml, en supposant que la diffusion est asynchrone (un deuxième message peut-être diffusé, même si tous les abonnés n'ont pas reçu le premier message).
3. Expliquer comment modifier le système pour faire une diffusion synchrone.

5 Sémantique d'Entrelacement

Voici un programme $P[i]$:

```
temp[i] := x;  
temp[i] := temp[i] + 1;  
x := temp[i]
```

où x et les $\text{temp}[i]$ sont des variables initialisées à 0. On suppose que chaque ligne de $P[i]$ est atomique.

On appelle P_n la mise en parallèle des n programmes $P[i]$ pour i de 0 à n :

$$P[0] \parallel P[1] \parallel \dots \parallel P[n-1] \parallel P[n]$$

1. Quelles sont les valeurs possibles de x après l'exécution de P_n , selon la sémantique d'entrelacement.
2. Donner tous les entrelacements possibles pour P_2 et, pour chacun, la valeur finale de x .
3. En supposant que tous les entrelacements de P_n ont la même probabilité d'apparaître lors d'une exécution, donner la distribution de probabilité des valeurs finales de x après l'exécution de P_n .